Name: Mohamed Aakif S

SERVICENOW ITSM IMPLEMENTATION LIFECYCLE

Phase 1: Brainstorming & Ideation Phase

Architectural Blueprint & Requirements Rationalization: Client Scripts vs. UI Policies

Target Table: Incident (`incident`)

Document Stage: Phase 1 of 5 (Ideation)

Platform Scope: Core ITSM & Service Operations

Author / Role: Technical Architecture Specialist

Governance: Declarative-First Policy

Target Client: Desktop, Agent Workspace, Mobile

---

1. Problem Statement & Executive Summary

Incident Management workflows are vulnerable to inconsistent input quality, high agent cognitive load, and incomplete transition records. While server-side business rules ensure authoritative persistence, client-side controls govern the immediate user experience. Unstructured or unguided incident records introduce service-level tracking issues, prolong Mean Time to Resolution (MTTR), and create bottlenecks during queue triage.

Core Objective: Establish a structured, clean client-side interaction layer on the ServiceNow `incident` table. This phase evaluates business requirements, eliminates unnecessary procedural scripting, and creates a clear separation of responsibility between **UI Policies** (declarative, configuration-based UI states) and **Client Scripts** (dynamic, scripted client logic).

2. Requirements Ideation & Technical Mapping

Following ServiceNow development governance, requirements are evaluated against the *Configuration before Customization* standard. UI Policies are chosen for direct state-to-field attributes, whereas Client Scripts are reserved for dynamic calculations, asynchronous external looks, and submission gatekeeping.

Req ID	Operational Scenario	Desired Form Behavior	Mechanism	Technical Rationale
REQ-01	State transitions to `Resolved` (State = 6)	Make `Close code` and `Close notes` mandatory and visible on the form.	UI Policy	Declarative condition. Supports automated state reversal if state moves back to Active.
REQ-02	Priority evaluates to `1 - Critical`	Display a high-visibility warning banner and make the `Workaround` field mandatory.	UI Policy + Action	Triggered purely on calculated field condition without complex logic requirements.
REQ-03	Caller field modified or selected	Dynamically retrieve caller VIP status, phone, and department; alert if VIP.	Client Script (onChange)	Requires asynchronous server retrieval (via `GlideAjax`) to avoid form lockup.
REQ-04	Category set to `Software`	Dynamically render specific software CI lookups and application-specific subcategories.	UI Policy	Standard visibility toggle driven solely by local choice field value.
REQ-05	Incident Submission (`onSubmit`)	Prevent submission if Urgency is `High` but Description has fewer than 30 characters.	Client Script (onSubmit)	Involves string validation, character counting, and conditional `return false` .

Req ID	Operational Scenario	Desired Form Behavior	Mechanism	Technical Rationale
REQ-06	Initial Form Initialization	Lock system-audited fields (`number` , `opened_by`) as permanently read-only.	**UI Policy**	Declarative enforcement without writing `g_form.setReadOnly()` routines.

3. Architecture & Technical Decision Framework

Architectural Decision Standard: Declarative First vs. Scripted

1. **UI Policy Priority:** UI Policies always take precedence when manipulating visibility (`setVisible`), mandatory state (`setMandatory`), or read-only status (`setReadOnly`) based on field conditions.
2. **Reverse if False Capability:** UI Policies natively reverse applied UI attributes when conditions fail, eliminating dozens of lines of error-prone `if/else` scripting.
3. **Execution Hierarchy:** In ServiceNow, client-side execution order during initial form render is: *onLoad Client Scripts → UI Policies → UI Policy Scripts*. Design decisions must prevent conflicting instructions across these layers.

When to Use UI Policies (Declarative)

- Pure attribute adjustments: Mandatory, Read-Only, Visible.
- No asynchronous external server data is needed.
- Conditions map directly to database fields.
- Cleaner upgrade path with minimal technical debt.

When to Use Client Scripts (Low-Code)

- Form submission gating via `onSubmit` validation.
- Fetching user/CI metadata asynchronously using `GlideAjax` .
- Manipulating choice list options dynamically (`g_form.addOption`).
- Validating string patterns (regex, phone numbers, email syntax).

4. Key Artifacts Identified for Subsequent Phases

UI Policy Backlog (Phase 2 Spec)

UIP-01: Incident State Resolution Governance
Trigger: `[State] [is] [Resolved]`
Actions: Enforce `close_code` and `close_notes` as mandatory/visible.

UIP-02: Read-Only System Auditing Fields
Trigger: Form Load (Global)
Actions: Set `number`, `sys_created_on`, `opened_by` to Read-Only.

UIP-03: Category-Driven CI Display
Trigger: `[Category] [is] [Hardware / Software]`
Actions: Display targeted CI reference fields dynamically.

Client Script Backlog (Phase 2 Spec)

CS-01: Asynchronous VIP & Caller Auto-Fill
Type: `onChange` (Field: `caller_id`)
Tech: Script Include + `GlideAjax` to fetch location and phone asynchronously.

CS-02: High-Urgency Detail Verification
Type: `onSubmit`
Tech: Ensures minimum description length (≥ 30 chars) and disallows dummy text.

CS-03: Category / Subcategory Dependency Tuning
Type: `onChange` (Field: `category`)
Tech: Cleanses and populates dynamic contextual options via `g_form.removeOption`.

5. Risk Analysis & Quality Engineering Guidelines

- **Race Conditions & Contention:** Never configure a Client Script and a UI Policy that manipulate the exact same field attribute simultaneously, as unpredictable execution orders will lead to intermittent UI defects.
- **Performance Degradation:** Synchronous server queries (e.g., `gr.query()` in client scripts) lock the agent's browser tab. All client scripts interacting with the database must strictly leverage `GlideAjax` with `getXMLAnswer()` or `g_scratchpad` populated by display business rules.
- **Workspace & Service Portal Compliance:** Ensure all script artifacts are registered with **UI Type = All** rather than legacy Desktop-only mode, guaranteeing seamless operation across ServiceNow Workspace.
- **Strict Prohibition of DOM Manipulation:** Bypassing platform APIs via `document.getElementById` or jQuery syntax is strictly forbidden to preserve future release upgradeability and Vancouver/Washington DC compatibility.

6. Deliverable Sign-Off & Phase 2 Transition Criteria

Phase 1 Approval & Next Steps:

With functional scenarios, technical mappings, and risk assessments defined, the ideation phase is complete. Proceed to **Phase 2: Design & Prototyping**, which will formalize exact dictionary field schemas, GlideAjax Script Include prototypes, client script functions, and configuration parameters.